Why String Is Immutable

9 min read Updated 28 Jul 2026 Topic 12 of 65

Why it matters

"Why is `String` immutable?" is one of the few interview questions where a complete answer requires four different areas of knowledge — security, hashing, concurrency and memory — and where a weak answer ("so it can be shared") is obviously incomplete. It is asked constantly for exactly that reason. It is also load-bearing: immutability is why strings can be pooled, why they are safe map keys, and why the entire `StringBuilder` class exists.

The concept, simply

A `String` object's characters can never change after construction. Every method that appears to modify a string — `concat`, `replace`, `toUpperCase`, `trim`, `substring` — returns a *new* `String` and leaves the original untouched. The class is `final`, its internal array is `private final`, and it is never handed out.

JAVA Copy

```java
String s = "hello";
s.toUpperCase();                // creates "HELLO" — and throws it away
System.out.println(s);          // hello

s = s.toUpperCase();            // reassignment is what actually changed s
System.out.println(s);          // HELLO
// The STRING did not change. The VARIABLE was pointed at a different stri
```

How it works

There are four distinct reasons, and a strong answer names all four.

1. Security. Strings carry the arguments of nearly every sensitive API in the JDK — file paths, class names, database URLs, socket hosts, permission names. If strings were mutable, a caller could pass a permitted value, wait for the security check to pass, then mutate the same object before it was used. That is a textbook *time-of-check-to-time-of-use* attack, and immutability eliminates it categorically: the value checked is the value used, because nothing can alter it in between.

2. Hash caching. Because the characters cannot change, a `String`'s hash code cannot change, so it is computed once and cached in a field. Strings are the most common map key in every Java program on Earth, and this makes every lookup after the first almost free. It also makes strings *safe* keys — Chapter 2's mutable-key trap can never happen with a `String`.

3. Thread safety. An immutable object has no state to race on, so a `String` can be shared across any number of threads with no synchronisation whatsoever. This is not merely convention: the memory model's `final`-field guarantee ensures every thread that sees a constructed `String` sees its contents fully initialised.

4. Pooling. Sharing one object between many references is only safe if nobody can modify it. Immutability is what makes the string pool possible — and the pool is what makes the enormous number of duplicate string literals in a typical program cost almost nothing.

Mechanically, immutability is enforced by three things together, and missing any one would break it: the field holding the characters is `private final`; no method ever exposes that array; and the class is `final`, so no subclass can add mutating behaviour or override methods to lie about the contents.

Code examples

What "returns a new String" costs, and why it matters:

JAVA

Copy

```java
String a = "hello";
String b = a.concat(" world");      // new object
String c = b.replace('l', 'L');     // another new object
```

```
// Three intermediate objects were created for one logical transformation.
// Harmless once; a real problem inside a loop (next topic).
```

The security argument, made concrete:

JAVA Copy

```
// Suppose String were mutable. This method could be defeated:
void loadConfig(String path) {
    if (!isAllowed(path)) throw new SecurityException(path);   // check
    // ── another thread mutates the caller's String object here ──
    readFile(path);                                            // use
}
// With real (immutable) Strings the value validated and the value read ar
// necessarily the same characters — there is no window to exploit.
//
// This is also why char[] is still recommended for PASSWORDS: you WANT to
// able to wipe it. Arrays.fill(pw, '\0') is possible; erasing a String is
// and it may linger in the pool and in heap dumps until GC.
```

Immutability as a design pattern you can copy — the same technique in your own classes:

JAVA Copy

```
public final class Money {                          // final: no subclass can
    private final long amountMinor;                 // final fields
    private final Currency currency;                // (and Currency is immut

    public Money(long amountMinor, Currency currency) {
        this.amountMinor = amountMinor;
        this.currency = Objects.requireNonNull(currency);
    }

    // "Mutators" return a new instance — exactly like String does.
    public Money plus(Money other) {
        if (!currency.equals(other.currency)) throw new IllegalArgumentExc
        return new Money(amountMinor + other.amountMinor, currency);
```

```
    public long amountMinor() { return amountMinor; }
}
// Free consequences: thread-safe, safe as a map key, safe to hand out,
// and impossible to leave in an inconsistent state.
```

Under the hood

Compact strings (Java 9+) are worth knowing because they contradict a fact people memorised. Before Java 9, `String` stored a `char[]` — two bytes per character, always. Since Java 9 it stores a `byte[]` plus a one-byte coder flag: strings whose characters all fit in Latin-1 use one byte each, and only strings needing wider characters use UTF-16. Since the overwhelming majority of strings in a typical application are ASCII, this roughly halved their memory footprint — one of the largest free wins in JVM history. So "a Java String uses 2 bytes per character" has been wrong since Java 9.

`substring` **used to share memory, and does not any more.** Before Java 7u6, `substring` returned a new `String` that *shared* the original character array with an offset and length. Taking a 10-character substring of a 10 MB string therefore retained all 10 MB — a notorious memory leak. Modern `substring` copies. It is a good answer to "has immutability ever caused problems?", because it shows you know the implementation changed and why.

Immutability is not free. Every transformation allocates. The JVM makes this cheap — young-generation allocation is a pointer bump and short-lived garbage is collected almost for free — but in a loop it becomes quadratic, which is the entire subject of the `StringBuilder` topic later in this chapter.

Common mistakes

- **Calling a String method and discarding the result.** `s.trim();` on its own does nothing. It must be `s = s.trim();`.
- **Believing reassignment is mutation.** `s = s + "!"` creates a new object; the original is unchanged and simply abandoned.

- **Storing passwords in a `String`.** You cannot erase it. Use `char[]` and wipe it, which is why `JPasswordField.getPassword()` returns one.
- **Assuming `final class` alone gives immutability.** A final class with a mutable field is not immutable. You need final fields, no exposed mutable internals, and defensive copies.
- **Answering only "so it can be shared".** True but partial. Security, hash caching and thread safety are the other three.
- **Saying a Java String is 2 bytes per char.** Outdated since Java 9's compact strings.

Interview lens

Q: "Why is `String` immutable in Java?"

A: Four reasons. Security — strings carry file paths, class names and connection URLs into sensitive APIs, and immutability removes any time-of-check-to-time-of-use window. Hash caching — the hash cannot change, so it is computed once and cached, which matters because strings are the most common map key. Thread safety — no mutable state means no synchronisation is ever needed. And pooling — sharing one object between many references is only safe if nobody can modify it.

Q: "How is that immutability actually enforced?"

A: Three things together. The character storage is a `private final` field, no method ever hands that array out, and the class itself is `final` so no subclass can add mutating behaviour or override methods to misreport the contents. Remove any one and the guarantee collapses.

Q: "What are the downsides?"

A: Every transformation allocates a new object, so concatenating in a loop is $O(n^2)$ in copying — which is what `StringBuilder` exists to solve. And you cannot erase a string's contents from memory, which is why passwords should be held in a `char[]` you can overwrite.

A: `StringBuilder` is mutable and deliberately not synchronised. `String` is thread-safe precisely because it is immutable — there is no state to race on. Note the two are different claims: immutable implies thread-safe, but thread-safe does not imply immutable, as `StringBuffer` shows.

Q: "How do you make your own class immutable?"
A: Make the class `final`; make every field `private final`; take no setters; copy any mutable object on the way in and on the way out; and return new instances from operations rather than mutating. In modern Java a `record` gives most of that by construction, though it still needs care if a component is itself mutable.

Q: "How many objects does `String s = new String("hi");` create?"
A: Potentially two: the literal `"hi"` goes into the pool when the class is loaded, and `new String( ... )` allocates a separate object on the heap that copies it. If the literal is already pooled from elsewhere, only the new object is created at that line. Which is the whole reason `new String( ... )` is a code smell — it exists to defeat pooling, and you almost never want that.

Cheat-card

- Immutable = characters never change. Every "mutator" **returns a new String**.
- Four reasons: **security · hash caching · thread safety · pooling**.
- Enforced by: `private final` storage + never exposed + `final` class.
- Hash is **cached** → strings are the ideal map key, and can never go stale.
- Cost: allocation per transformation → O(n²) in loops → use `StringBuilder`.
- Passwords → `char[]`, so they can be wiped.
- Java 9 **compact strings**: `byte[]` + coder, 1 byte per char for Latin-1.
- Pre-7u6 `substring` shared the array and leaked; it copies now.

QUICK CHECK — no marks, just for you.

Java Interview Kit

1. Which is NOT one of the reasons String is immutable?

- ◯ It allows the hash code to be computed once and cached
- ◯ It removes a time-of-check-to-time-of-use window in security-sensitive APIs
- ◯ It makes String operations faster than StringBuilder operations
- ◯ It makes the string pool safe, since one object can be shared by many references

Check answer

2. Why are passwords recommended to be held in a char[] rather than a String?

- ◯ char[] uses less memory
- ◯ String cannot hold non-ASCII characters
- ◯ char[] is encrypted automatically by the JVM
- ◯ A char[] can be overwritten with Arrays.fill once used; an immutable String cannot be erased and may linger in the pool and in heap dumps until collected

Check answer

3. How much memory does a Java String use per character on Java 9 and later?

- ◯ One byte for Latin-1 content, two for anything wider — compact strings store a byte[] plus a coder flag
- ◯ Always 4 bytes, for full Unicode
- ◯ It depends on the platform's default charset

Check answer

Previous
static, final and Initialisation Order

Next
The String Pool and intern()

LEARN MORE

ARTICLES & DOCS

- String — API docs
  docs.oracle.com
- Compact Strings (JEP 254)
  openjdk.org

VIDEOS



String Immutability & String Pool Explained with Examples | Java Interview Guide
CodeSnippet



Understanding String Immutability in Java From Scratch | String Pooling and Interning
Riddhi Dutta

Hand-picked references for this lesson.